Object-Oriented Programming (OOP) in Java

Complete Syllabus with Topic Descriptions

1. Introduction to OOP

Object-Oriented Programming (OOP) is a programming paradigm based on objects that contain both data and methods. It helps in building modular, reusable, and scalable software systems by modeling real-world entities.

2. Java Basics for OOP

Understanding Java program structure, JVM, JDK, JRE, data types, variables, control statements, and methods. These fundamentals are required before learning object-oriented concepts.

3. Classes and Objects

A class is a blueprint for creating objects, while an object is an instance of a class. Objects contain state (fields) and behavior (methods).

4. Constructors

Constructors initialize objects when they are created. Java supports default, parameterized, and constructor overloading.

5. Encapsulation

Encapsulation bundles data and methods together and restricts direct access to data using access modifiers. It improves data security and modularity.

6. Abstraction

Abstraction hides implementation details and exposes only essential features using abstract classes and interfaces.

7. Inheritance

Inheritance allows a class to inherit properties and methods from another class, promoting code reuse and hierarchy.

8. Polymorphism

Polymorphism allows methods to behave differently based on objects. It includes method overloading and method overriding.

9. Interfaces

Interfaces define contracts that classes must implement. They support multiple inheritance and abstraction in Java.

10. Abstract Classes

Abstract classes provide partial abstraction and may include both abstract and concrete methods.

11. Method Overloading & Overriding

Overloading allows multiple methods with the same name but different parameters. Overriding allows subclass methods to replace superclass methods.

12. Access Modifiers

Public, private, protected, and default access modifiers control visibility of classes and members.

13. Packages in Java

Packages organize related classes and interfaces into namespaces, improving modularity and maintenance.

14. Exception Handling

Exception handling manages runtime errors using try, catch, finally, throw, and throws mechanisms.

15. File Handling

Java provides file and stream classes to read and write data to files for persistent storage.

16. Collections Framework

The Java Collections Framework provides data structures like List, Set, Map, and Queue for efficient data handling.

17. Multithreading

Multithreading allows concurrent execution of threads, improving performance in applications.

18. Java Memory Management

Understanding stack, heap memory, and garbage collection in Java applications.

19. Lambda Expressions & Functional Interfaces

Modern Java supports functional programming using lambda expressions and functional interfaces.

20. Mini Project / Application Development

Applying OOP principles to build real-world applications such as management systems or learning platforms.